Student Management System Rebuild Guide

This document explains the codebase, how the pieces fit together, and the exact execution order needed to recreate and run the project.

Backend: Node HTTP server

Frontend: static HTML/CSS/JS

Automation: Playwright

Storage: backend/students.json

1. What This Project Does

The project is a small student management system with three layers:

Login flow

The login page accepts any non-empty name, stores it in localStorage under `studentSystemUser`, and navigates to the dashboard.

Dashboard flow

The dashboard greets the saved user and shows the number of students loaded from the API.

Students flow

The students page supports create, edit, delete, refresh, and status messaging through `/api/students`.

Automation flow

Playwright tests cover login, navigation, and full CRUD behavior from the browser UI.

2. Project Structure

Path	Role
backend/server.js	HTTP server, static file hosting, and student CRUD API.
backend/students.json	Persistent JSON data store for students.
frontend/login.html	Login screen.
frontend/dashboard.html	Summary page with student count and navigation.
frontend/students.html	CRUD UI for students.
frontend/script.js	Shared browser logic for login, dashboard, and students pages.
frontend/style.css	Visual styling for the static UI.
automation/tests/*.spec.js	Playwright tests for login, UI, and CRUD behavior.
automation/pages/*.js	Page objects used by Playwright tests.

3. Runtime Architecture

- The backend uses Node's built-in `http` module, not Express.
- The server listens on `PORT` if set, otherwise `3000`.
- `/` serves `frontend/login.html`.
- `/dashboard.html`, `/students.html`, CSS, and JS are served as static files from the frontend folder.
- `/api/students` supports `GET`, `POST`, `PUT`, and `DELETE`.
- Student data is stored in `backend/students.json` and created automatically if missing.
- The frontend keeps the current user in localStorage and uses fetch calls to communicate with the API.

> The app is not a SPA. It uses separate HTML pages and one shared script file to switch behavior based on `body[data-page]`.

4. Backend Behavior

Server startup

- `backend/server.js` creates the HTTP server.
- It ensures `backend/students.json` exists before accepting requests.
- It starts listening on `http://localhost:3000` by default.

API behavior

- `GET /api/students` returns the full array.
- `POST /api/students` creates a new student and auto-assigns the next numeric id.
- `PUT /api/students/:id` merges incoming fields into the student record while preserving the id.
- `DELETE /api/students/:id` removes the record and returns the deleted student.
- `GET /health` returns `{ ok: true }`.

Important execution detail

Because the frontend calls the API with relative paths, the browser must open the pages through the backend server, not by double-clicking the HTML files directly.

5. Frontend Behavior

Login page

- User enters a name and clicks Continue.
- The name is stored in localStorage under `studentSystemUser`.
- The app navigates to `dashboard.html`.

Dashboard page

- The heading becomes `Welcome, [name]`.
- The student count is fetched from `/api/students`.

Students page

- The form can add a new student or edit an existing one.
- The table renders name, email, age, and action buttons.
- Delete uses a confirmation dialog before calling the API.
- Refresh reloads the table from the backend.
- Status text is shown in `#statusMessage`.

6. Exact Execution Steps

Use these steps in order to recreate the working project state:

Step	Command	What it does
1	`npm install` in the project root	Installs Playwright dependencies used by the root and automation workspace.
2	`cd backend && npm install`	Prepares the backend package, even though it only uses Node built-ins for runtime.
3	`cd backend && npm start`	Starts the HTTP server on `http://localhost:3000`.
4	Open `http://localhost:3000/`	Loads the login page through the backend.
5	Enter a name and click Continue	Saves the user in localStorage and opens the dashboard.
6	Open Students from the dashboard	Loads the CRUD page and fetches the student list.
7	Create, edit, refresh, and delete a student	Exercises the API and updates `backend/students.json`.
8	`npx playwright test`	Runs the automated browser checks from the root config.

> If you want the browser test runner UI, use `npx playwright test --ui`. If browser binaries are missing, run `npx playwright install` first.

7. Rebuild Checklist

- Create the folder layout exactly as shown above.
- Implement the Node server with static file hosting and student CRUD endpoints.
- Build three HTML pages that all load one shared script and one shared stylesheet.
- Keep login state in localStorage and reuse it on the dashboard.
- Make the Students page call the API with relative paths so it works from the backend origin.
- Add Playwright page objects to keep the tests readable and stable.
- Run the backend first, then run the browser tests against it.

8. Commands To Remember

```
cd d:\Testtingproject
npm install
cd backend
npm start

# in another terminal
cd d:\Testtingproject
npx playwright test

# optional UI runner
npx playwright test --ui

# install browser binaries if needed
npx playwright install
```